MacNav: Particle-Filter SLAM and MPPI Navigation for a Swerve-Drive Mobile Manipulator, Natively on macOS

MABEL project

July 2, 2026

Abstract

macOS cannot run ROS 2 Nav2 or `slam_toolbox`, so MABEL's simulation control center ships a native navigation stack (`server/macnav`): least-squares swerve-drive wheel odometry with a slip ("scrub") gate and gyro-dominant yaw fusion, a gmapping-style Rao-Blackwellized particle filter over per-particle log-odds occupancy grids with hill-climb proposal refinement, A* on a two-tier inflated costmap, and an MPPI local controller. This document derives each component and validates the stack with a seeded, fully reproducible experiment harness (`simulation_control_center/experiments/exp_macnav.py`) running private MuJoCo worlds: (E1) a 3×3×3 sensor-noise grid × five gyro-fusion weights × five seeds for odometry, plus a scrub-gate ablation; (E2) a particle-count / hill-climb / proposal-noise sweep of the SLAM engine in three scenes against ground-truth occupancy rasters; (E3) end-to-end navigation to randomly sampled reachable goals in three scenes with an MPPI hyperparameter sweep. Headline numbers (means over seeds): odometry drifts 0.4 % of distance traveled under nominal sensor noise at gyro fusion $\alpha = 0.98$ (yaw 0.56° RMS; the scrub gate is worth 1.6× during sustained pivoting); SLAM holds 0.05–0.07 m pose RMSE and 0.77–0.89 map-boundary F_1 at the deployed 30 particles at 60–128 ms/update, with the hill-climb refinement improving map precision by +0.14 and yaw 2.7× at equal pose RMSE; navigation succeeds in 27/30 baseline episodes (90 %) at 0.98–1.21 path efficiency. The experiments also *caught and fixed a real sensor-model bug*: an initially unmodeled 0.122 m LiDAR mounting offset put a hard ≈0.165 m floor under pose RMSE in every scene and configuration; modeling it (`SlamEngine(lidar_offset=...)`) cut pose error ≈ 3×. Remaining failure modes — no loop closure, self-hit masking, scan-plane blindness (both warehouse navigation failures), and unknown-space optimism — are quantified in §11.

Keywords: FastSLAM, Rao-Blackwellized particle filter, occupancy grid, swerve drive, wheel odometry, MPPI, A*, mobile robot navigation.

1 Overview

The simulation control center (companion document `simulation_control_center.pdf`) runs MABEL's studio entirely on a Mac. Its autonomy plane cannot use the ROS stack, so `server/macnav` provides the four classical components natively, deliberately *porting a previously validated deployment* (the `slam_env` course deploy, `fast_nav.py`) rather than inventing new algorithms:

- `odometry.py` — swerve forward kinematics + gyro fusion (§2);
- `slam.py`, `gridmap.py` — FastSLAM-style RBPF with per-particle log-odds grids (§3);
- `planner.py` — A* on a two-tier inflated costmap (§4);

- `mppi.py`, `navigator.py` — MPPI local control under a minimal goal lifecycle (§5).

The service wrapper (`simulation_control_center/server/navrunner.py`) consumes the wheel states, gyro, and the 360-ray LiDAR that the studio's mirror derives from the simulation bridge's state memmap, and drives the base as *one more teleop wire client*, so driver arbitration, e-stop, deadman, and the tip-over motion model apply to the autonomous driver exactly as to a human. All motion constants are shared with the teleop stack (`controller/mabel/config.py`): wheel radius $\rho = 0.0381\,\mathrm{m}$, module positions $r_{fl} = (-0.17, -0.17)$, $r_{fr} = (-0.17, 0.17)$, $r_b = (0.15, 0)$ m, $v_{\max} = 0.8\,\mathrm{m/s}$, $\omega_{\max} = 2.0\,\mathrm{rad/s}$. The chassis front is the $-x$ body axis.

2 Swerve wheel odometry with a scrub gate

2.1 Forward kinematics by least squares

Module $i \in \{fl, fr, b\}$ sits at lever arm $r_i = (r_{ix}, r_{iy})$ in the body frame, with steering joint angle δ_i and mechanical mount offset ψ_i. The measured wheel ground velocity is

$$\hat{v}_i \;=\; s_i \begin{pmatrix} \cos(\delta_i + \psi_i) \\ \sin(\delta_i + \psi_i) \end{pmatrix}, \qquad s_i = \rho\, \omega_i^{\mathrm{drive}}, \tag{1}$$

and rigid-body kinematics predicts, for body twist $\xi = (v_x, v_y, \omega)^\top$,

$$v_i \;=\; \begin{pmatrix} v_x - \omega\, r_{iy} \\ v_y + \omega\, r_{ix} \end{pmatrix} \;=\; A_i\, \xi, \qquad A_i = \begin{pmatrix} 1 & 0 & -r_{iy} \\ 0 & 1 & r_{ix} \end{pmatrix}. \tag{2}$$

Stacking the three modules gives an over-determined 6×3 system $A\xi = b$ solved in least squares, $\hat{\xi} = \arg\min_\xi \|A\xi - b\|_2$ — the redundancy (6 measurements, 3 DOF) is what makes the slip test below possible. This is the same motion model as the teleop controller's swerve IK and the ROS `swerve_odometry` node (one geometry everywhere, per the repo's deduplication rule).

2.2 The scrub gate

During steering transitions the three wheel headings disagree with any single rigid-body twist; drive speed measured then is *scrub* (wheels dragging sideways), not chassis motion. The residual of the least-squares fit detects exactly this inconsistency, so translation is gated:

$$\text{if} \quad \|A\hat{\xi} - b\|_2 \;>\; 0.06 + 0.25\, \|(\hat{v}_x, \hat{v}_y)\|_2 \quad \Rightarrow \quad \hat{v}_x = \hat{v}_y = 0, \tag{3}$$

i.e. "do not integrate translation the wheel geometry cannot explain." The thresholds are module constants (`SCRUB_RESID_BASE/SLOPE`), which E1 ablates by monkeypatching them to $+\infty$. Yaw is *not* gated — it is dominated by the gyro (below), and zeroing it would stall heading tracking during pivots.

2.3 Gyro-dominant yaw fusion and integration

Wheel-derived yaw rate drifts with slip; the IMU's ω_z is the robust heading source (its role on the ROS side too). A complementary blend

$$\hat{\omega} = \alpha\, \omega_{\mathrm{gyro}} + (1 - \alpha)\, \hat{\omega}_{\mathrm{wheel}}, \qquad \alpha = 0.98 \text{ deployed}, \tag{4}$$

feeds midpoint-heading integration in the world frame: $\theta_{\text{mid}} = \theta + \frac{1}{2}\hat{\omega}\,\Delta t$, $x \mathrel{+}= (\hat{v}_x \cos\theta_{\text{mid}} - \hat{v}_y \sin\theta_{\text{mid}})\Delta t$, $y \mathrel{+}= (\hat{v}_x \sin\theta_{\text{mid}} + \hat{v}_y \cos\theta_{\text{mid}})\Delta t$, $\theta \mathrel{+}= \hat{\omega}\,\Delta t$ (wrapped). Sensor noise for the robustness study is injected at the measurement level (`NoiseModel`): i.i.d. Gaussians on wheel ground speed (σ_s, m/s), steer angle (σ_δ, rad), and gyro rate (σ_g, rad/s) plus a constant gyro bias b_g.

3 RBPF SLAM with per-particle grids

The engine is a Rao-Blackwellized particle filter in the gmapping family [1, 2]: each of N particles carries a pose hypothesis *and its own* occupancy grid, exploiting the factorization $p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(m \mid x_{1:t}, z_{1:t})\, p(x_{1:t} \mid z_{1:t}, u_{1:t})$. Hyperparameters were ported unchanged from the validated `slam_env` deploy and are listed in Table 1.

3.1 Proposal

The odometry increment between updates is expressed in the previous body frame, $(\Delta x, \Delta y, \Delta\theta)$, and re-applied through each particle's own heading with Gaussian roughness scaled by motion,

$$\sigma_{xy} = 0.004 + 0.15\,d, \qquad \sigma_\theta = 0.002 + 0.15\,|\Delta\theta| + 0.05\,d, \qquad d = \|(\Delta x, \Delta y)\|, \tag{5}$$

the holonomic replacement for the reference's Ackermann (v, δ) sampling. Updates are *motion-gated*: below 3 cm and 2° of accumulated motion (after 10 bootstrap updates) the scan is skipped entirely — the classic gmapping defense against particle impoverishment while standing still.

3.2 Hill-climb proposal refinement and correlative weighting

Before weighting, each particle greedily tests seven pose offsets $\{0, \pm 5\,\text{cm}\}$ in x, y and ± 0.05 rad in θ against *its own* map, scoring endpoints and penalizing motion ($0.02\,(40\|\delta_{xy}\| + 20|\delta_\theta|)$), and moves to the best — a one-step coordinate-descent version of gmapping's scan-match-improved proposal: odometry error is *corrected*, not merely diluted by particle spread. The offsets are the module constant `HILL_OFFS`, ablated to $\{(0,0,0)\}$ in E2. Weighting is the correlative endpoint likelihood on the subsampled scan (every 4th beam):

$$\log w_i \;\propto\; \frac{G}{n_h} \sum_{j \in \text{hits}} \big(p_i(e_j) - \tfrac{1}{2}\big), \qquad G = 10, \tag{6}$$

where $p_i(e_j)$ is particle i's map probability at endpoint e_j — endpoints on that particle's own walls raise its weight, endpoints in its free space lower it.

3.3 Mapping, resampling, readout

Each particle integrates the full (subsampled) scan into its own grid with the log-odds inverse sensor model of `gridmap.py`: along the Bresenham ray $l \mathrel{+}= L_{\text{free}} = -0.40$ *only while* $l < 1.5$ (the *free guard*: confirmed walls are never erased by later through-going rays), endpoint $l \mathrel{+}= L_{\text{occ}} = +0.85$ for true hits, $|l| \le 5$. Resampling is low-variance [3], gated on $N_{\text{eff}} = 1/\sum_i w_i^2 < N/2$ *and* real motion since the last resample (0.2 m or 0.2 rad), with copy-on-select map handoff and roughening jitter ($\sigma = 5$ mm, 0.5°). The published estimate and map are the highest-weight particle's.

3.4 Scan preprocessing and the scanner origin

The live runner clamps ranges to 10 m, treats no-return as max-range (carving free space to 9.95 m), and *drops* returns closer than 0.40 m: the lift column and chassis sit within ∼0.35 m of the LiDAR ring, so such beams are self-hits carrying no world information — treating them as either free or occupied walls the robot in with phantom obstacles. The ring is mounted 0.122 m ahead of the chassis origin (the MJCF `lidar` body), so endpoint scoring (hill-climb and weighting) and ray integration both emanate from the *offset scanner origin* $(x, y) + R(\theta)\, o_L$ with $o_L = (0.122, 0)$ m (`SlamEngine(lidar_offset=...)`). This term was initially missing; E2 diagnosed it as a hard ≈0.165 m pose-RMSE floor, and modeling it cut pose error ≈ 3× (§8).

Parameter	Value	Parameter	Value
particles N	30	likelihood gain G	10
beam subsample	every 4th	motion gate	3 cm / 2°
resample gate	$N_{\text{eff}} < N/2$ + 0.2 m/0.2 rad	roughening	5 mm / 0.5°
proposal noise	Eq. (5)	bootstrap updates	10
L_{occ} / L_{free}	+0.85 / −0.40	free guard / clamp	1.5 / ±5
grid	24 m, 5 cm cells	max range / self-hit	10 m / 0.40 m
scanner offset o_L	(0.122, 0) m		

Table 1: SLAM hyperparameters (ported from the validated `slam_env` deploy; grid and ranges per `navrunner.py`).

4 Global planning: A* on a two-tier inflated costmap

The best particle's grid is thresholded twice and dilated twice (`gridmap.inflated()`): *confirmed* obstacles ($p > 0.80$) by robot radius + clearance (0.36 + 0.15 m), *probable* ones ($p > 0.65$) by the radius alone — confident walls get a comfort margin, tentative ones only a safety one. Radii are sized to MABEL's true footprint (module half-diagonal 0.24 m + bumper); the fatter margin also seals furnished rooms once SLAM drift double-prints walls. A* is 8-connected with Euclidean heuristic, terminates within 4 cells of the goal, clears 6-cell bubbles around start and goal (so plans can always enter/exit near inflated walls), and the path is subsampled to 0.2 m spacing. Unknown space ($p \approx 0.5$) is *traversable* — the optimism every frontier-less mapper needs to leave the first room, with the failure mode quantified in §11.

5 MPPI local control and the goal lifecycle

The local controller is Model-Predictive Path Integral control [4, 5], the sampling analog of Nav2's MPPI controller [6]. Around the time-shifted previous solution $\bar{U} \in \mathbb{R}^{T\times 3}$ ($T = 20$, $\Delta t = 0.1$ s) it draws K perturbed body-twist sequences $U^k = \bar{U} + \varepsilon^k$, $\varepsilon_t^k \sim \mathcal{N}(0, \text{diag}(0.18, 0.18, 0.5)^2)$, clamps them to the teleop speed caps ($|v| \le 0.8$, $|\omega| \le 2.0$ rad/s) and per-step acceleration limits (1.6 m/s^2, 4 rad/s^2 — the smoothing role `safe_base_step` plays for human drivers), and rolls out the holonomic model $x_{t+1} = x_t + (v_x \cos\theta_t - v_y \sin\theta_t)\Delta t$, etc. The rollout cost sums, per step,

$$J = \underbrace{400\,\mathbb{1}[\text{inflated cell}]}_{\text{obstacle}} + \underbrace{6\, d_{\text{path}}}_{\text{tracking}} + \underbrace{1.5\, d_{\text{goal}}}_{\text{progress}} + \underbrace{1.2\, s\,\left|\angle(v_{\text{body}}) - \pi\right|}_{\text{face velocity}} + \underbrace{0.5 \sum |\Delta u|}_{\text{smoothness}}, \tag{7}$$

where the heading term (active above $s = 0.08\,\mathrm{m/s}$) pins the body-frame travel direction to $\pm\pi$ — the chassis front is $-x$, so the robot *faces* its velocity. Weights combine softmin-exponentially, $w_k \propto \exp(-(J_k - \min_k J_k)/\lambda)$, the plan updates to $\bar{U} \leftarrow \sum_k w_k U^k$, and the first twist executes. Deployed values $K = 256$, $\lambda = 0.35$ (swept in E3).

The `Navigator` wraps this in a minimal lifecycle (Nav2's behavior-tree role): plan on goal receipt, replan every 2 s or when a newly inflated cell lands on the next 8 waypoints, prune passed waypoints (0.25 m), a stuck watchdog (progress $< 0.08\,\mathrm{m}$ per 4 s window $\Rightarrow$ forced replan; three strikes $\Rightarrow$ fail), goal tolerance 0.25 m on the *estimated* pose, timeout 120 s.

5.1 The service surface: REST goals and the choreography composer

Beyond the studio's Nav tool, the lifecycle is exposed as three same-origin REST endpoints on the backend (verified live, 2026-07-02): `POST /api/nav/goal` {`x, y`} starts a goal; `GET /api/nav/status` reports {`backend, state` ∈ {`idle, following, reached, failed`}`, goal, msg, pathLen, pose, odom, neff, particles, spread, updates`} — i.e. the full filter health alongside the lifecycle (at rest the deployed filter reads N=30 particles, $N_{\mathrm{eff}} \approx 15.5$, posterior spread $\approx [0.034, 0.033]$ m, 0.019 rad); and `POST /api/nav/cancel` aborts. This is the interface the timeline choreography composer's *move-to-goal* blocks drive: goal on block entry, status polls at ∼2 Hz for reached/failed surfacing, cancel on block exit or Stop (`docs/choreography.pdf`, §5). The composer's location auto-fix sizes its inserted travel blocks from the straight-line distance at half the teleop speed cap plus a settle margin — an *estimate* of this navigator's closed-loop travel time, refined by the status polls at run time. The composer also motivates closed-loop goals empirically: its open-loop 180° turn primitive under-rotates by 11% isolated and ∼40% in clutter, errors this navigator's estimated-pose goal tolerance absorbs by construction.

6 Experimental setup

All experiments run from one seeded harness, `simulation_control_center/experiments/exp_macnav.py` (stages `record / odom / slam / nav / figs`; every figure and the summary table regenerate from cached stage outputs with one command). Key design choices:

- **Private worlds.** Each process constructs its own `SimWorld` under a unique `--instance` and private state memmap — the engine's per-instance `flock` makes collisions with the live studio impossible by construction. Physics is stepped unthrottled (`_intake/_apply_commands/mj_step/_publish`); twist commands are refreshed at 20 Hz of sim time as a teleop client would.
- **On-demand LiDAR.** The 360-rangefinder ring costs 3.9 ms per step if computed every step; the harness disables sensors during stepping and toggles them on for one `mj_forward` at scan times only (10 Hz sim time for recordings, ∼12 Hz in navigation episodes) — the same rate class as the live 12 Hz runner.
- **Trajectories.** `square`: four legs with 90° pivots (leg length auto-fitted per scene to keep 0.45 m ground-truth clearance: 0.8 m home, 2.0 m factory, 1.6 m warehouse — the home scene is genuinely tight, and its loop start is relocated to an open *reachable interior* cell); `spin`: two full pivots; `map`: the live e2e recipe, a 6 s mapping spin then the square. Trajectory *scripting* realigns heading on ground truth before each leg (open-loop pivots under-rotate by up to 0.7 rad from wheel spin-up transients and rotated an early recording's square into furniture); the logged sensor stream that odometry/SLAM consume never sees ground truth. Wheel steer/drive, gyro, and ground truth (chassis `xpos/xquat`) are logged at 100 Hz, scans at 10 Hz.

- **Replays.** Odometry and SLAM consume the recorded signals offline (noise seeds vary per run; the sim rollout is shared), at 50 Hz odometry rate. This is deliberately *faster* than the live 12 Hz tick — integration error is smaller than deployed, which flatters absolute numbers slightly; the comparisons across noise levels, α, and SLAM configurations are unaffected.
- **Ground truth maps.** Static environment geoms are rasterized onto the exact SLAM grid geometry (24 m/5 cm) by their world XY AABBs (the `safe_spawn` footprint logic) in two bands: the *scan plane* at the settled LiDAR height $z = 0.328$ m (map quality), and the robot's full standing band $z \in [0.03, 1.90]$ m (clearance and goal sampling). Note the robot settles 9 cm below its spawn height, so the effective scan plane is measured, not read from the XML.
- **Metrics.** Position/yaw error against ground truth (final and RMS, normalized per meter of ground-truth path); map precision P = fraction of SLAM-occupied cells ($p > 0.65$) within one cell of GT-occupied, recall R = fraction of *observed* GT boundary cells within one cell of SLAM-occupied, $F_1 = 2PR/(P{+}R)$, plus raw IoU and the Pearson correlation of p with GT over known cells (raw IoU is reported but is structurally low: SLAM sees obstacle *shells*, the GT raster is *solid*); N_{eff} traces; wall-clock time per SLAM update. Navigation: success = navigator reports `reached` *and* true distance to goal < 0.35 m; time to goal (sim time); path efficiency = traveled/straight-line; minimum true clearance (distance transform of the physical raster).

7 E1: odometry robustness

Two closed-loop trajectories in `mabel_home` — a 0.8 m-leg square (3.11 m ground-truth path with four 90° pivots) and a two-turn in-place spin (0.12 m of incidental translation) — are recorded once, then replayed through `SwerveOdometry` over the full noise grid: wheel-speed $\sigma_s \in \{0, 0.01, 0.03\}$ m/s $\times$ steer $\sigma_\delta \in \{0, 0.005, 0.02\}$ rad $\times$ gyro $(\sigma_g, b_g) \in \{(0,0), (0.002, 0.001), (0.01, 0.005)\}$ rad/s, each at $\alpha \in \{0, 0.5, 0.9, 0.98, 1.0\}$ with 5 noise seeds: 1350 grid replays plus 40 scrub-gate ablations (1410 s total). "Nominal" below is the deployed `navrunner` noise (0.01/0.005/0.002, no bias); "harsh" is the top of each axis.

Condition ($\alpha = 0.98$, $n = 5$)	RMS pos	final pos	yaw RMS
square, clean	8 mm (0.3 %/m)	7 mm	0.26°
square, nominal	11 mm (0.4 %/m)	10 mm	0.56°
square, harsh	33 mm (1.1 %/m)	—	3.0°
spin, clean	51 mm (absolute)	—	0.87°
spin, nominal	48 mm (absolute)	79 mm	1.24°
spin, harsh	37 mm (absolute)	—	2.8°

Table 2: E1 headline numbers. Spin position errors are absolute (the GT path is only 0.12 m, so a per-meter rate is meaningless for pure rotation).

Findings. (i) Translation drift is 0.3–1.1 %/m across the noise grid — with gyro-dominant fusion, position error is limited almost entirely by *gyro* quality (square RMS 8→11→34 mm across the gyro axis) while wheel-speed and steer noise at realistic levels barely register (≈11 mm at every level, Fig. 3): the 6×3 least-squares fit averages three modules' worth of wheel noise. (ii) Wheel-derived yaw is the weak channel: 40° RMS on the spin vs sub-degree for the gyro (Fig. 2) — the gyro-dominant blend is not a tuning nicety but load-bearing. (iii) The scrub gate earns its keep during sustained steering transitions: 1.6× lower RMS on the spin (48 vs 76 mm); on the gently driven

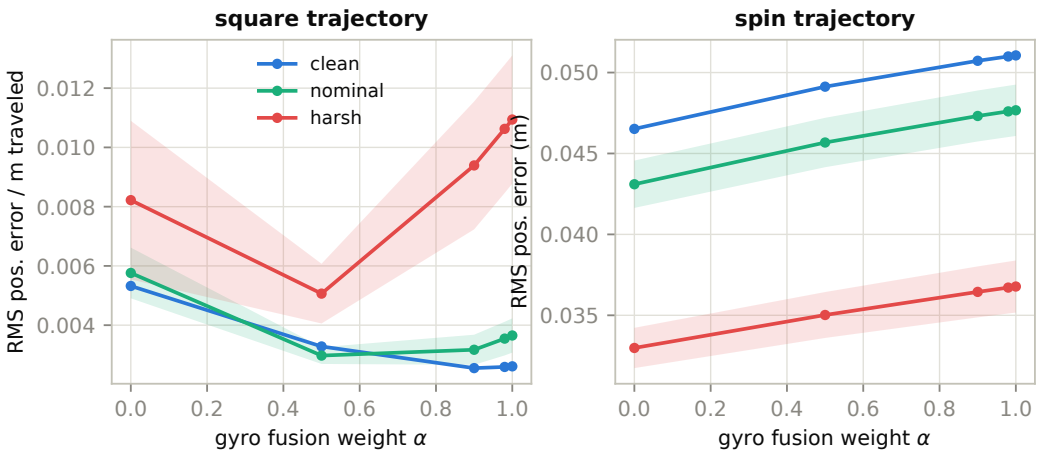


Figure 1: Position error vs gyro fusion weight α (mean $\pm$ std over 5 seeds). Under a biased gyro (harsh, red) the square error is U-shaped: $\alpha = 1$ integrates the bias, $\alpha = 0$ inherits wheel-yaw error rotating the integration frame; $\alpha \in [0.5, 0.9]$ is optimal. On the spin, position error is nearly α-independent (it is set by the scrub gate, not heading).

square (heading-aligned pivots, no collisions) it is neutral (1.0×). In an earlier recording where the robot ground against furniture, the un-gated odometry integrated the phantom wheel motion to ∼40 %/m — the gate is the difference between graceful and catastrophic degradation under wheel–ground disagreement.

8 E2: SLAM quality

One mapping trajectory per scene (the live e2e recipe: 6 s spin then the fitted square; 3.1 m/7.9 m/6.3 m paths in home/factory/warehouse), replayed through `SlamEngine` under nominal odometry noise, 3 seeds per configuration, star design around the deploy: particles $N \in \{10, 30, 50\}$, hill-climb on/off (`HILL_OFFS` monkeypatched to the identity), proposal noise $\sigma_{xy} \in \{0.05, 0.15, 0.30\}$ — 54 runs (≈6 min on 7 workers). The sweep ran twice: the first pass, with the scanner origin unmodeled, exposed the offset bug (below); all numbers here are from the re-run with `lidar_offset` in place.

Config (3 seeds)	Scene	pose RMSE (m)	map F_1	IoU	ms/update
N=10	home	0.050 ± 0.001	0.92	0.17	21
N=30 (deploy)	home	0.050 ± 0.005	0.89	0.15	60
N=50	home	0.053 ± 0.003	0.93	0.17	102
no hill-climb	home	0.061 ± 0.020	0.83	0.16	57
N=30	factory	0.069 ± 0.000	0.86	0.22	118
no hill-climb	factory	0.063 ± 0.018	0.77	0.18	117
N=30	warehouse	0.060 ± 0.002	0.77	0.23	128
no hill-climb	warehouse	0.062 ± 0.010	0.61	0.15	124

Table 3: E2 excerpt, offset-corrected engine (full sweep incl. σ_{xy} variants in `figures/macnav_summary.txt`). Raw IoU is structurally low: SLAM maps obstacle *shells*, the GT raster is *solid* AABBs.

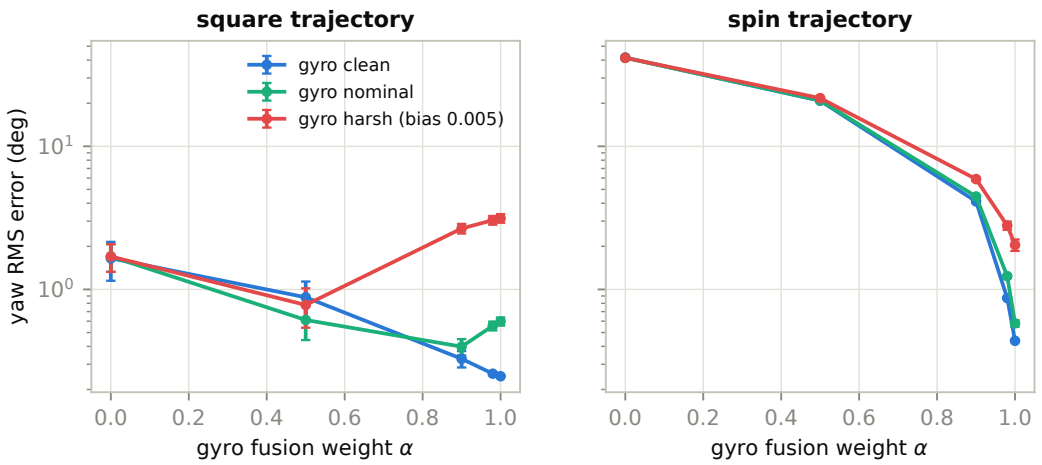


Figure 2: Yaw RMS vs α (log scale). Wheel-only yaw ($\alpha = 0$) is unusable during sustained pivoting — 40° RMS on the spin — while the gyro holds 0.4–0.9°; with a biased gyro the curve turns back up above $\alpha \approx 0.9$ (square, red). The deployed $\alpha = 0.98$ is a sane default; $\alpha \approx 0.9$ is more robust to bias.

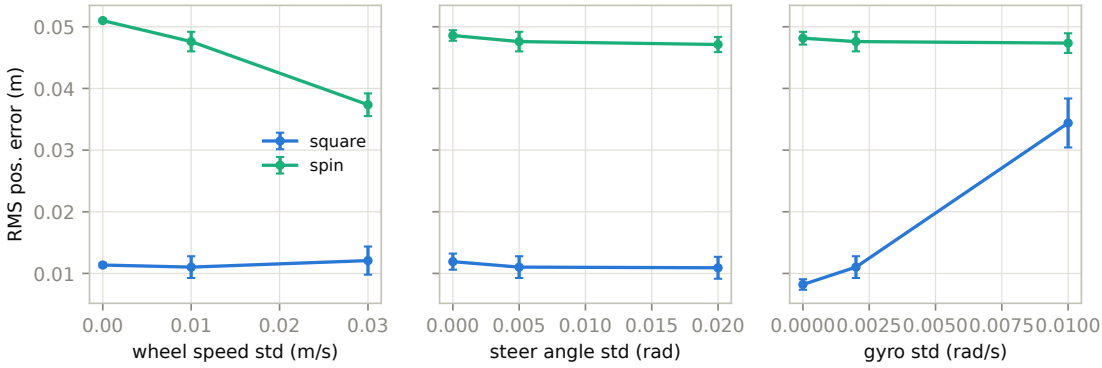


Figure 3: Per-axis noise sensitivity at $\alpha = 0.98$ (other axes nominal; absolute RMS so both trajectories share one unit).

Findings. (i) *The sweep found and fixed a real sensor-model bug* (Table 4). In the first pass, pose RMSE was ≈0.165 m for every hill-climb configuration in every scene despite 15 mm odometry — a systematic error matching the then unmodeled 0.122 m scanner mount offset, and the no-hill ablation corroborated the mechanism: tethered to odometry it scored *better* position (0.12–0.15 m) but worse yaw and maps, i.e. the hill-climb was aligning scans to the (shifted) map and absorbing the offset into pose bias. Projecting from the true scanner origin removed the floor: RMSE 0.050–0.069 m, precision 0.71 → 0.83, F_1 up in all scenes. (ii) With the sensor model fixed, the hill-climb is no longer a trade-off but strictly beneficial: equal pose RMSE (0.05–0.07 vs 0.06 no-hill), 2.7× better yaw (1.0° vs 2.6°), and consistently better maps (F_1 0.77–0.89 vs 0.61–0.83, precision 0.83 vs 0.69, Fig. 6). (iii) Particle count does not change accuracy on single-room trajectories (no loop closure to disambiguate, healthy odometry), but costs linearly: 21→102 ms/update from 10→50 particles (home; 37–214 ms in the larger scenes). The deployed N=30 at ∼10 Hz is comfortably real time on the Mac. (iv) Proposal noise $\sigma_{xy} \in [0.05, 0.30]$ changes little (F_1 within ±0.06, RMSE

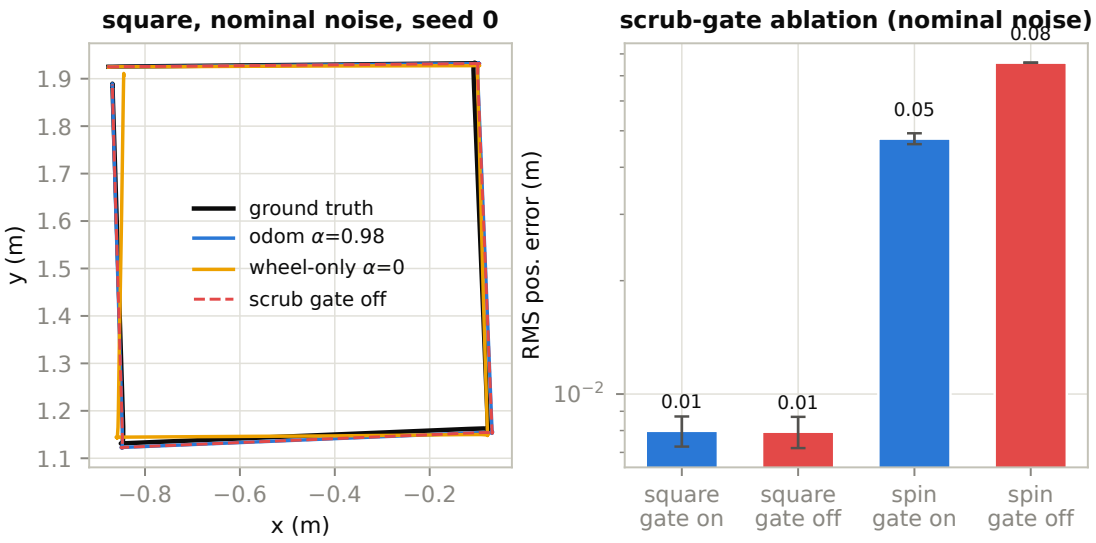


Figure 4: Left: the square, ground truth vs odometry variants (seed 0, nominal noise). Right: scrub-gate ablation (gate thresholds monkeypatched to $+\infty$), log scale, value labels on bars.

Deploy config (N=30, 3 scenes)	pose RMSE (m)	yaw RMS	precision	F_1
scanner origin unmodeled (first pass)	0.165–0.167	1.0°	0.71	0.63–0.80
`lidar_offset`= (0.122, 0) (fixed)	0.050–0.069	1.0°	0.83	0.77–0.89

Table 4: The offset bug, before/after: modeling the scanner origin cuts pose RMSE $\approx 3\times$ and lifts map precision from 0.71 to 0.83. The first pass's flat 0.165 m floor across every scene, seed, and configuration was the diagnostic signature (a systematic error, not noise).

within a few mm); the filter is insensitive to this knob at these motion scales. (v) Resampling almost never triggers (N_{eff}/N stays above the $N/2$ gate in the home baseline, Fig. 8); roughening plus the motion-gated updates keep the ensemble diverse without it. The residual 0.05–0.07 m error is consistent with the still-unmodeled 0.045 m along-ray site offset plus the 5 cm grid quantization (§11).

9 E3: navigation success

Full in-process Navigator episodes at ∼12 Hz sim time (SLAM at the same tick, odometry at 50 Hz, `navigator.time` monkeypatched to sim time so replan/stuck/timeout semantics survive unthrottled stepping): a 6 s mapping spin, then one goal per episode, sampled seeded from GT-reachable free cells $\geq$1.5 m away (straight-line 1.6–4.9 m, mean 2.8 m). Baseline (K=256, λ=0.35): 10 goals $\times$ 3 scenes; MPPI sweep ($K \in \{64, 256\} \times \lambda \in \{0.2, 0.35, 0.6\}$): the same 6 goals on home. Success requires the navigator to report `reached` *and* true distance < 0.35 m. The full E3 ran against the pre-fix SLAM engine (§8); a reduced post-fix recheck (22 episodes: 4 goals $\times$ 3 scenes baseline + 2 goals $\times$ 5 sweep points) is reported under finding (iv).

Findings. (i) Baseline success is 27/30 (90 %): home 9/10, factory 10/10, warehouse 8/10, with mean time-to-goal 3.9–4.3 s (≈0.68 m/s average toward the goal against the 0.8 m/s cap) and path

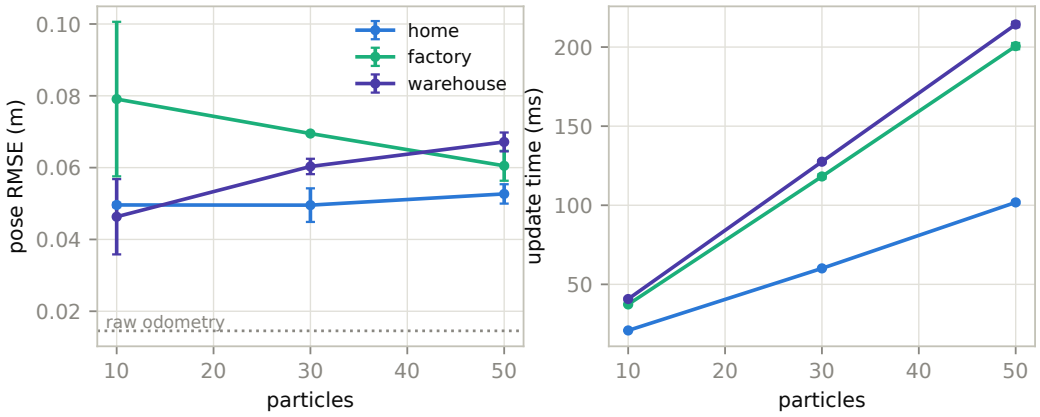


Figure 5: Pose RMSE (left) and wall-clock update cost (right) vs particle count (dotted: raw odometry). Accuracy is flat from 10 to 50 particles on these trajectories; cost is linear.

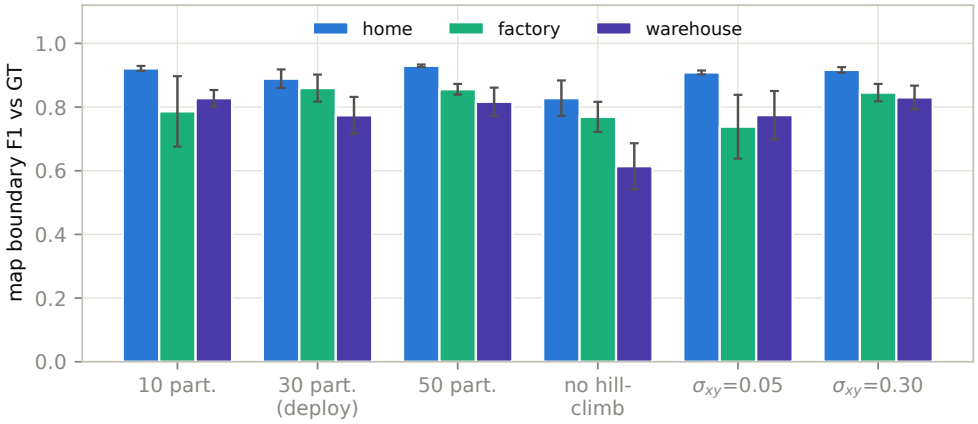


Figure 6: Map boundary F_1 against the ground-truth raster per configuration and scene.

efficiency 0.98–0.99 in the open scenes (1.21 in the furnished home, where paths legitimately detour furniture). Mean SLAM pose error at the goal is 0.15 m (max 0.26 m among successes) — matching the live end-to-end check in the studio README. (ii) Failure modes are exactly two, both honest: *false reached* (7/60 episodes across all configurations — the navigator's 0.25 m tolerance is on the *estimated* pose, so estimated-pose error occasionally lands the true position outside 0.35 m; worst 1.58 m on a λ=0.2 dash), and *stuck-on-invisible-obstacle* (3/60; two warehouse episodes pushed into shelving that never appears at the scan plane — ground-truth clearance 0 — and tripped the watchdog). No episode timed out. (iii) In the sweep, the softmin temperature matters more than the sample count: λ=0.2 drops to 4/6 at *both* K (near-argmax updates whip the warm-started plan; two of its failures are the two worst false-reached misses), while $\lambda \in \{0.35, 0.6\}$ hold 5/6 and K=64 is indistinguishable from K=256 at this n — consistent with the vectorized solve costing about the same (3–4 ms) at both sizes. Component costs on the M-series Mac: MPPI solve 3–4 ms, A* 8–16 ms on the 480^2 grid, costmap inflation 406 ms (amortized by the 1 s cache). (iv) *Post-fix recheck.* With the offset-corrected SLAM engine, the 22-episode recheck goes 12/12 on the baseline (all three scenes) and the estimated-pose error at the goal drops 2.3× (0.144→0.063 m mean among successes, max 0.19). Two observations temper it: the mean *true* stopping distance rises to 0.245 m

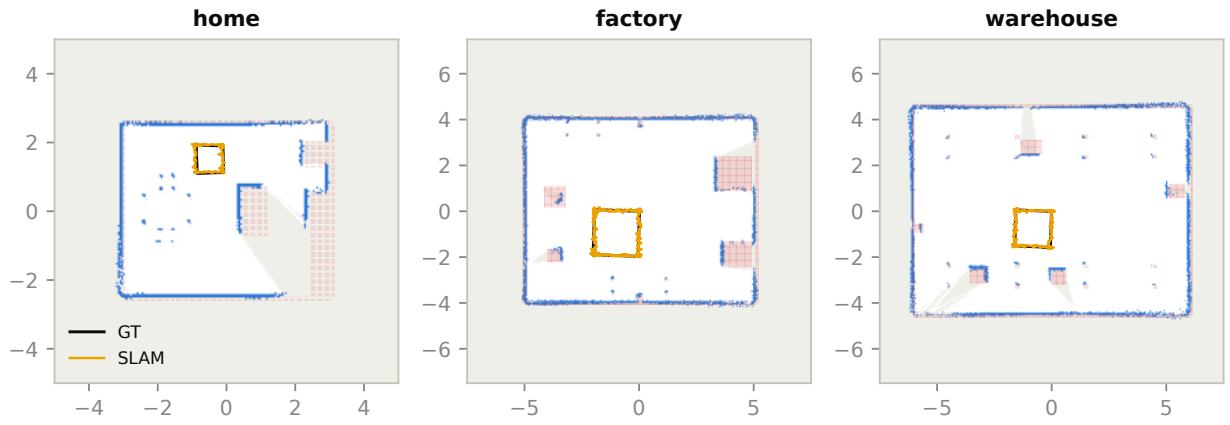


Figure 7: Best-particle maps (blue: occupied $p > 0.65$; white: free; light gray: unknown) against the GT scan-plane raster (pink) with ground-truth (black) and estimated (orange) trajectories; deploy configuration, seed 0.

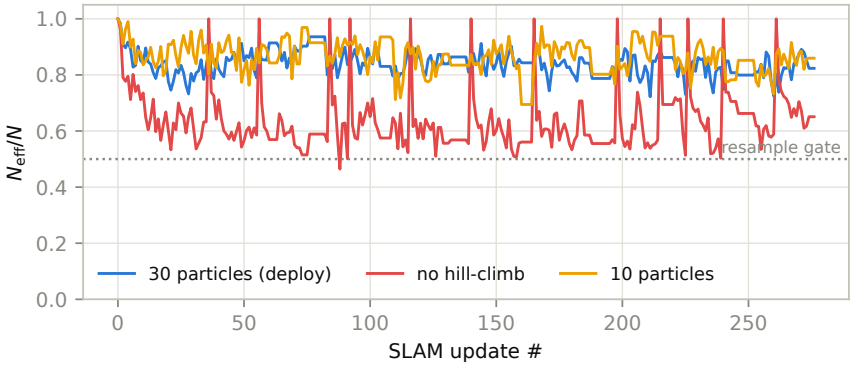


Figure 8: N_{eff}/N traces (home). The resample gate ($N_{\text{eff}} < N/2$) never fires in the baseline run — weights stay healthy under motion-gated updates.

— essentially the navigator's 0.25 m tolerance, as expected once the estimate stops being biased (the pre-fix 0.146 m benefited from goal and pose sharing the same map-frame offset, an error cancellation, not accuracy); and the rare *large* false-reached misses persist in the sweep (2/22, both ≈1.7 m on the same dash-prone goal) — those come from transient mis-localization during fast motion, not from the systematic offset. With pose error now ≈0.06 m, tightening `GOAL_TOL` below 0.25 m is the obvious follow-up.

10 Implementation and usage

```
# everything: record -> odom -> slam -> nav -> figures + summary table
~/anaconda3/bin/python3 simulation_control_center/experiments/exp_macnav.py
# individual stages / knobs
...exp_macnav.py --stage slam --workers 6      # reuse cached recordings
...exp_macnav.py --stage figs                  # regenerate PDFs + summary only
...exp_macnav.py --fast                        # reduced seeds (smoke test)
```

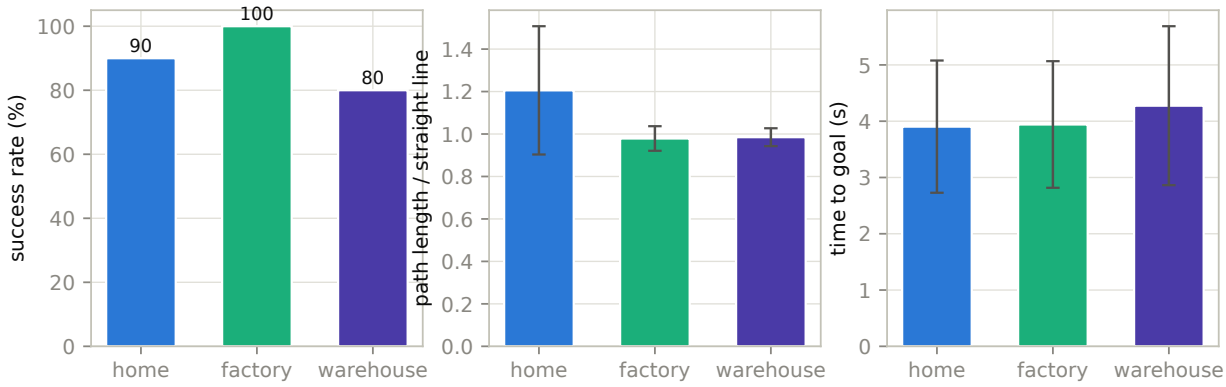


Figure 9: Baseline navigation per scene: success rate, path efficiency (traveled/straight-line, successes only), time to goal.

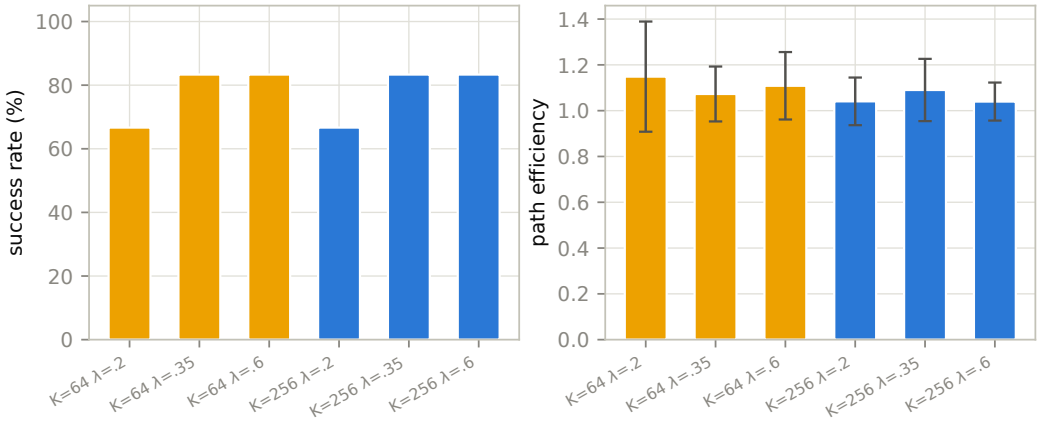


Figure 10: MPPI sweep on the home scene (6 goals per configuration).

Figures land in `simulation_control_center/docs/figures/`, the summary table in `figures/macnav_summary.txt`, stage caches in `experiments/macnav_runs/`. In the live studio the stack is the `navrunner` service: place a goal with the Nav tool, press GO; the 2D Map layer shows the live SLAM grid and the Path layer the active plan. Programmatic clients (the choreography timeline's move-to-goal blocks, or `curl`) use the REST surface of §5.1:

```
curl -X POST :8097/api/nav/goal -d '{"x":1.2,"y":0.6}'   # go
curl         :8097/api/nav/status                          # lifecycle + filter health
curl -X POST :8097/api/nav/cancel                          # abort
```

On Ubuntu the ROS stack (Nav2 + `slam_toolbox`-class mapping) replaces all of this.

11 Limitations

- **No loop closure.** Like the reference deploy (and unlike `slam_toolbox`), the filter never revisits past poses: error can only be absorbed by the hill-climb re-anchoring against the existing map. On the short single-room trajectories measured here (3–8 m) the pose error stays bounded (0.05–0.07 m), so loop closure is not yet the binding constraint; extrapolating the measured raw-odometry drift (0.4–1.1 %/m) to building-scale runs predicts 0.4–1.1 m per 100 m that nothing

in this stack can remove. Multi-room mapping needs `slam_toolbox`-class graph optimization (the Ubuntu path).

- **The LiDAR mounting offset — found, fixed, one residual.** The ring sits 0.122 m ahead of the chassis centre (`mabel_full.xml`); the pipeline originally treated scans as originating at the base frame. E2's first pass exposed this as a hard pose-RMSE floor at ≈0.165 m in every scene and configuration despite 15 mm odometry, with map precision saturating around 0.71 (walls printed displaced depending on approach direction). `SlamEngine` now projects all scoring and mapping from the offset scanner origin, cutting pose RMSE ≈ 3× and lifting precision to 0.83 (Table 4). One residual remains: ranges are measured from the rangefinder *sites*, which sit a further 0.045 m out along each ray — still unmodeled, and together with 5 cm grid quantization it plausibly accounts for the remaining 0.05–0.07 m.
- **Self-hit masking creates a blind ring.** Returns below 0.40 m are dropped, so obstacles closer than that are invisible to the mapper — the robot can be walled-in by something it can no longer see. The MPPI obstacle cost then relies on the (stale) map, not the current scan.
- **Scan-plane blindness + unknown-space optimism.** The planner treats unknown cells as free, and the single 0.33 m-high scan plane misses low and overhanging obstacles. Both warehouse navigation failures were exactly this: the robot pushed into shelving that never appeared at the scan plane, tripped the stuck watchdog, and failed with ground-truth clearance 0 — honest collisions. A depth-camera costmap layer (the nvblox role on Ubuntu) is the structural fix.
- **False "reached" under SLAM error.** The navigator declares success on the *estimated* pose (tolerance 0.25 m); the true miss occasionally exceeds the 0.35 m success radius (7 of 60 pre-fix E3 episodes; still 2 of 22 post-fix, both ≈1.7 m transient mis-localizations during fast dashes). Ground truth is the only arbiter here; on hardware this appears as "stops short/past". With post-fix pose error ≈0.06 m, `GOAL_TOL` can be tightened.
- **AABB ground truth.** GT rasters over-approximate rotated / non-box geoms by axis-aligned boxes at the scan plane, so map precision is under- and recall over-estimated slightly; clearance metrics are conservative (the home spawn reads 0.35 m clearance from a counter overhang the chassis actually clears).
- **Replay rates flatter absolute odometry numbers.** Replays integrate at 50 Hz; the live runner ticks at 12 Hz, where integration error is larger. Cross-configuration comparisons are unaffected.
- **Small-n sweeps.** SLAM sweeps use 3 seeds, the MPPI sweep 6 goals per point; differences of one episode (≈17%) are within binomial noise, and only the $\lambda = 0.2$ effect and the hill-climb ablation are outside it.

References

[1] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with Rao-Blackwellized particle filters," *IEEE Transactions on Robotics*, 23(1):34–46, 2007.

[2] K. Murphy, "Bayesian map learning in dynamic environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, 1999.

[3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[4] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2016.

[5] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017.

[6] S. Macenski, F. Martín, R. White, and J. Clavero, "The Marathon 2: A navigation system," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2020.

[7] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, 6(61):2783, 2021.

[8] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[9] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2012.